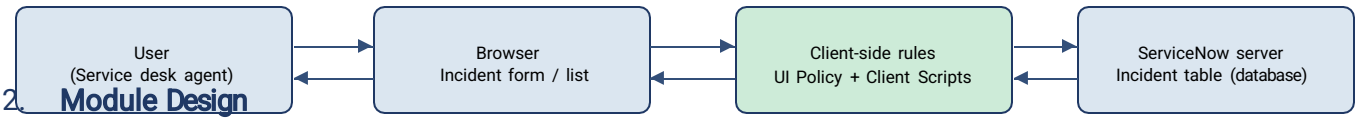
Project Design Phase

Project Title	implement client script and ui policy
Team Name / ID	s.ezhilamanibala,lathika,ilakiya,saravanan,durai balaji
Team Members	team3
Date	30.09.2026

1. System Architecture

- A client-side solution on the ServiceNow platform: the browser loads the Incident form, the UI Policy and Client Scripts run in the browser, and only valid records are sent to the ServiceNow server and saved in the Incident table.

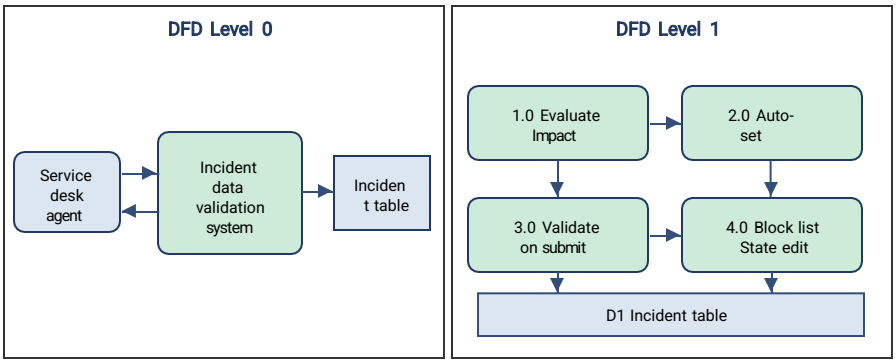


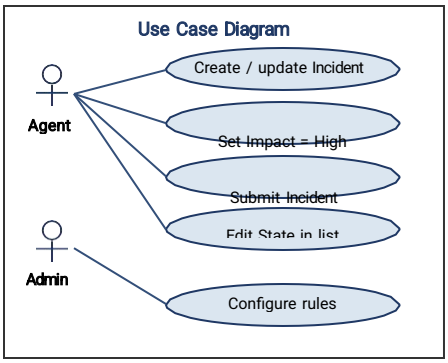
2. Module Design

Rules run in the browser before the record is sent to the server

Module	Description	Owner
UI Policy: High Impact Control	Makes Assignment group mandatory and Urgency read-only when Impact = High; reverses when false	[Name]
onChange Client Script	Sets Urgency to 1 - High and shows an info message when Impact changes to High	[Name]
onSubmit Client Script	Blocks save if Impact = High and Assigned To is empty; shows an error	[Name]
onCellEdit Client Script	Blocks State edits in list view with an alert	[Name]

3. Data Flow / UML Diagrams





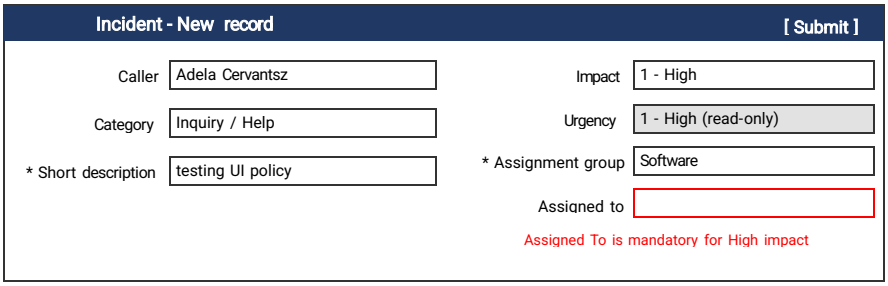
Impact evaluation, Urgency auto-set, submit validation and list-edit blocking work on the Incident table. Agents create and submit incidents; the admin configures the UI Policy and Client Scripts.

4. Database Design

- **Incident** (number, caller_id, category, short_description, impact, urgency, priority, state, assignment_group, assigned_to). Priority is calculated from Impact and Urgency.
- **Relationships:** caller_id and assigned_to reference User (sys_user); assignment_group references Group (sys_user_group). No schema changes are made.

5. UI / UX Design

Standard ServiceNow Incident form (no custom UI). Mock-up of the form with the rules applied when Impact = High:

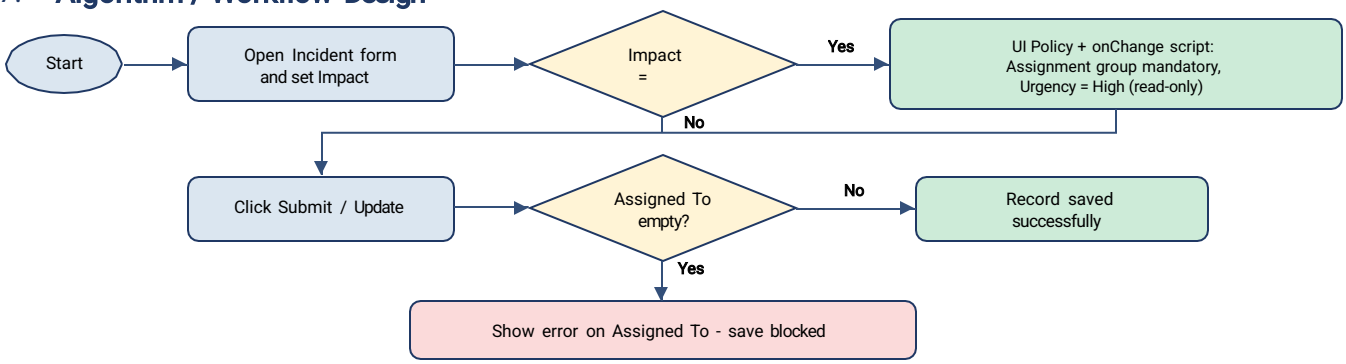


Design tools: ServiceNow form view for the screens; diagrams in this document were drawn for the project.

6. Technology Stack

- **Frontend:** ServiceNow Incident form and list (UI Policy, Client Scripts in JavaScript) | **Backend:** ServiceNow platform (no custom server code)
- **Database:** ServiceNow instance tables (incident) | **Tools:** Personal Developer Instance, Chrome / Edge browser and developer console

7. Algorithm / Workflow Design



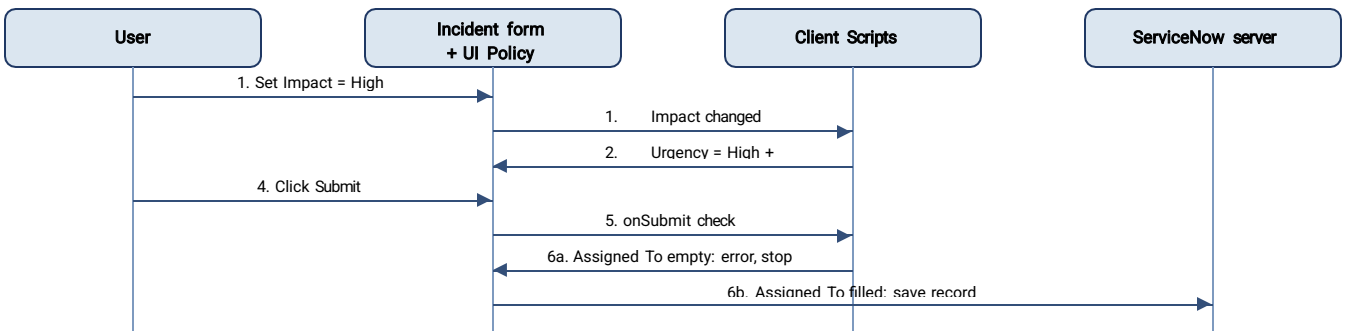
- **Input:** Impact, Assigned To and other Incident fields entered by the user. **Processing:** UI Policy applies rules, scripts auto-set Urgency and validate on submit. **Output:** a saved valid incident, or an error message and a blocked save.

8. Security & Error Handling Design

- **Authentication and access control:** users must log in to ServiceNow; only admin / configuration roles can create or change UI Policies and Client Scripts.
- **Data validation:** mandatory fields through UI Policy and save-time checks through onSubmit; State edits from the list are cancelled.
- **Error handling:** the error box appears on the Assigned To field, an info message explains the Urgency change and an alert explains the list-edit block. Nothing is saved until data is valid.

9. Design Decisions & Justification

- Client-side controls were chosen over server-side rules because they give instant feedback, are conditional and reversible, and need no extra cost or server code. UI Policy handles simple field behaviour, and Client Scripts handle logic and validation.
- **Scalability and maintenance:** rules use standard, well-named records and can be extended to other tables or backed by server-side Business Rules later.

10. Notes / Space for Diagrams

Sequence diagram of the main scenario: